

CTD / Glider Data Analysis

IIMS Sensor Processing Project — WS 2025/26

Dataset: SLOCUM G3 glider, Atlantic Ocean

Task: Identify structure of density distribution

Sections:

1. Setup & data loading
2. Data inspection
3. Preprocessing & derived variables
4. Profile plotter (single dive)
5. Section plotter (2D transect)
6. T-S diagram
7. Buoyancy frequency N^2

```
In [1]: import numpy as np
import pandas as pd
import xarray as xr
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import cmoccean
import gsw
from scipy.ndimage import gaussian_filter1d

plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})
```

1 — Data loading

column	unit	description
time	datetime64	UTC timestamp
lat	°N	latitude
lon	°E	longitude
depth	m (positive down)	depth
pressure	bar	measured pressure
temperature	°C	in-situ temperature
conductivity	S/m	conductivity

```
In [2]: # — SET YOUR FILE PATH —
NC_FILE = 'data/GliderIFM16_CTD.nc'

# — Open and inspect —
```

```

ds = xr.open_dataset(NC_FILE)
print("Dimensions:", dict(ds.sizes))
print("Variables :", list(ds.data_vars))
for var in ds.data_vars:
    print(f" - {var}: [{ds[var].attrs.get('units', 'No units')}]")

```

Dimensions: {'time': 313538}

Variables : ['lat', 'lon', 'depth', 'conductivity', 'temperature', 'pressure']

```

- lat: [decimal degrees]
- lon: [decimal degrees]
- depth: [m]
- conductivity: [S/m]
- temperature: [degC]
- pressure: [Bar]

```

2 — Data inspection

```

In [3]: ds['lat'] = ds['lat'].where(xr.ufuncs.abs((ds['lat'])) <= 90) # Replace
print("Missing latitude values:", ds['lat'].isnull().sum())

ds['lon'] = ds['lon'].where(xr.ufuncs.abs((ds['lon'])) <= 180) # Replace
print("Missing longitude values:", ds['lon'].isnull().sum())

ds = ds.dropna(dim='time')
ds = ds.dropna(dim='time')

print("Latitude range:", ds['lat'].min().item(), "to", ds['lat'].max().item())
print("Longitude range:", ds['lon'].min().item(), "to", ds['lon'].max().item())

df = pd.DataFrame({
    'time':      pd.to_datetime(ds['time'].values, unit='s'),
    'lat':       ds['lat'].values.astype(float),
    'lon':       ds['lon'].values.astype(float),
    'depth':     ds['depth'].values.astype(float),
    'pressure':  ds['pressure'].values.astype(float),
    'temperature': ds['temperature'].values.astype(float),
})

cond = ds['conductivity'].values.astype(float)

# conductivity in S/m but gsw.SP_from_C expects mS/cm => ×10
cond = cond * 10
print("Conductivity converted from S/m to mS/cm")
df['salinity'] = gsw.SP_from_C(cond, df['temperature'], df['pressure'])
print("Salinity derived from conductivity via gsw.SP_from_C")

# Before dropna — how many real GPS fixes?
print("GPS fixes before dropna:", df['lat'].notna().sum())

# Drop rows with missing values
df = df.dropna(subset=['depth', 'temperature', 'salinity']).reset_index(drop=True)

# After dropna — how many survived?
df_clean = df.dropna(subset=['depth', 'temperature', 'salinity'])

print("GPS fixes after dropna:", df_clean['lat'].notna().sum())

```

```

df['lat'] = df['lat'].interpolate(method='linear')
df['lon'] = df['lon'].interpolate(method='linear')
df = df.sort_values('time').reset_index(drop=True)

# — Pressure unit guard: SLOCUM sometimes stores pressure in bar not dba
df['pressure'] *= 10
print("pressure converted from bar to dbar")

print(f"\nLoaded {len(df):,} measurements across ")

```

Missing latitude values: <xarray.DataArray 'lat' ()> Size: 8B
array(8927)

Attributes:

units: decimal degrees

Missing longitude values: <xarray.DataArray 'lon' ()> Size: 8B
array(8927)

Attributes:

units: decimal degrees

Latitude range: 46.625945001436065 to 46.997676659927734

Longitude range: -42.43710333163882 to -41.40202893895057

Conductivity converted from S/m to mS/cm

Salinity derived from conductivity via gsw.SP_from_C

GPS fixes before dropna: 304611

GPS fixes after dropna: 304611

pressure converted from bar to dbar

Loaded 304,611 measurements across

In [4]:

```
print(df.dtypes)
print(df[['depth', 'pressure', 'temperature', 'salinity', 'lon', 'lat']].c
```

time datetime64[ns]

lat float64

lon float64

depth float64

pressure float64

temperature float64

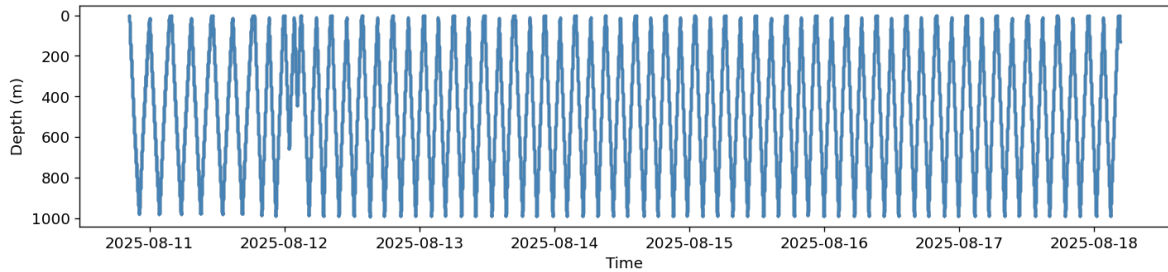
salinity float64

dtype: object

	depth	pressure	temperature	salinity	lon	lat
count	304611.000	304611.000	304611.000	304611.000	304611.000	304611.000
mean	489.432	489.995	5.539	35.063	-41.725	46.800
std	286.744	287.680	2.483	0.400	0.231	0.105
min	0.004	0.220	3.684	32.419	-42.437	46.626
25%	241.242	240.930	4.221	35.013	-41.882	46.706
50%	484.695	484.680	4.687	35.180	-41.645	46.742
75%	736.479	737.630	5.589	35.291	-41.568	46.886
max	991.735	994.750	18.733	35.914	-41.402	46.998

Glider Trajectory

```
In [5]: fig, ax = plt.subplots(figsize=(12, 3))
ax.scatter(df['time'], df['depth'], s=1, c='steelblue', alpha=0.3, rasterized=True)
ax.set_ylabel('Depth (m)')
ax.set_xlabel('Time')
ax.invert_yaxis()
plt.tight_layout()
```



```
In [6]: from matplotlib.colors import ListedColormap
import matplotlib.dates as mdates

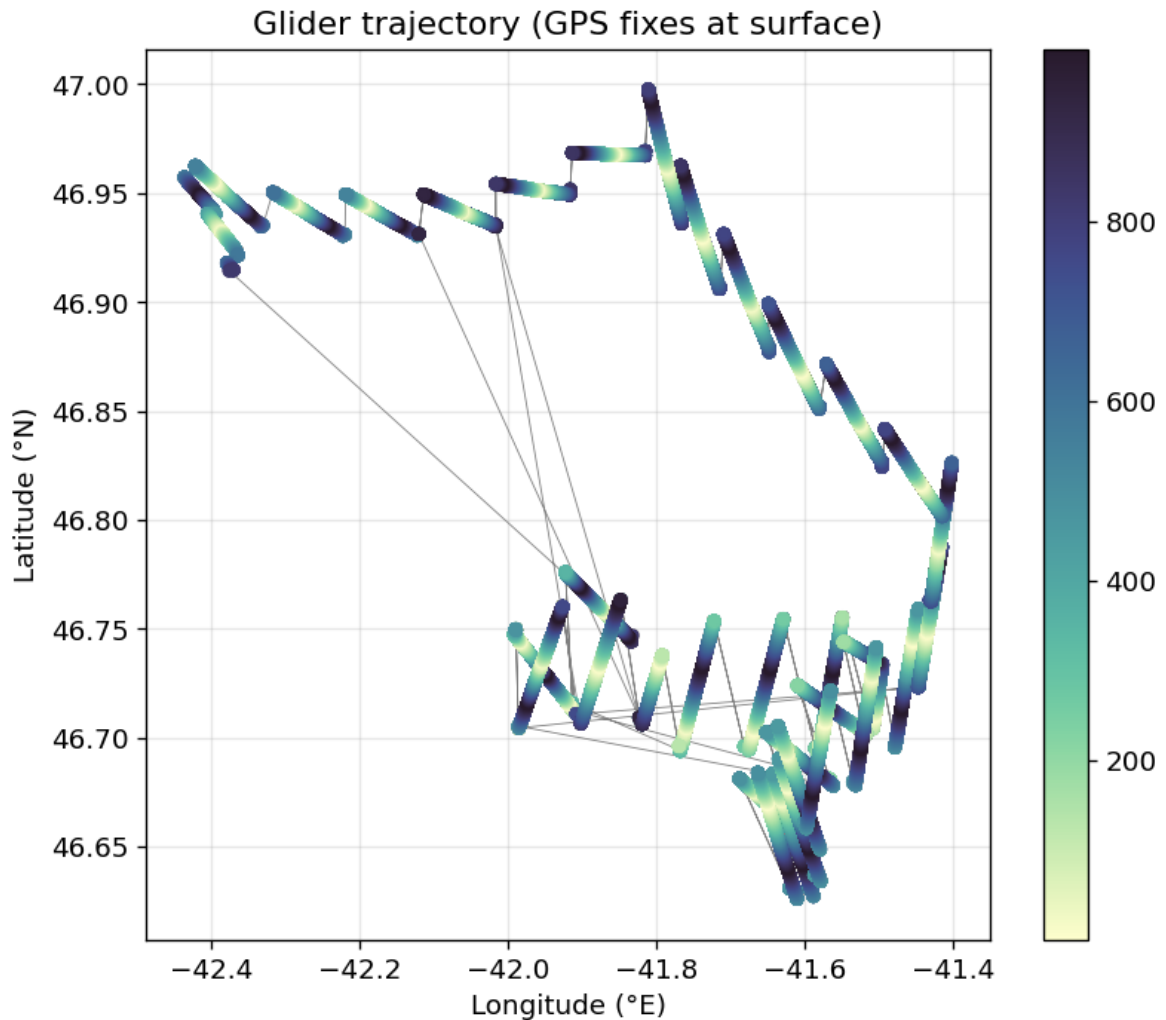
lat = df['lat'].values
lon = df['lon'].values
time = df['time'].values          # Unix seconds
depth = df['depth'].values

#changes = np.concatenate([[True], np.diff(lat) != 0])
#lat, lon, time = lat[changes], lon[changes], time[changes]

time_dt = pd.to_datetime(time, unit='s')
time_num = mdates.date2num(time_dt.to_pydatetime()) # float matplotlib needs

tab10 = plt.get_cmap('tab10')
cmap66 = ListedColormap([tab10(i % 10) for i in range(66)])

fig, ax = plt.subplots(figsize=(7, 6))
sc = ax.scatter(lon, lat, c=depth, cmap=cmocean.cm.deep, s=20, zorder=3)
ax.plot(lon, lat, lw=0.5, color='grey', zorder=2)
cbar = plt.colorbar(sc, ax=ax)
#cbar.ax.yaxis_date()
#cbar.ax.yaxis.set_major_formatter(mdates.DateFormatter('%b %d'))
ax.set_xlabel('Longitude (°E)')
ax.set_ylabel('Latitude (°N)')
ax.set_title('Glider trajectory (GPS fixes at surface)')
ax.grid(alpha=0.3)
plt.tight_layout()
```



```
In [7]: # Blank out lat/lon everywhere except at the surface
df['lat_clean'] = df['lat'].where(df['depth'] < 1)
df['lon_clean'] = df['lon'].where(df['depth'] < 1)

# Interpolate linearly between surface fixes
df['lat_clean'] = df['lat_clean'].interpolate(method='linear')
df['lon_clean'] = df['lon_clean'].interpolate(method='linear')

print("NaN remaining in lat_clean:", df['lat_clean'].isna().sum())
print("NaN remaining in lon_clean:", df['lon_clean'].isna().sum())
```

NaN remaining in lat_clean: 0

NaN remaining in lon_clean: 0

```
In [8]: from matplotlib.colors import ListedColormap
import matplotlib.dates as mdates

lat = df['lat_clean'].values
lon = df['lon_clean'].values
time = df['time'].values # Unix seconds

#changes = np.concatenate([[True], np.diff(lat) != 0])
#lat, lon, time = lat[changes], lon[changes], time[changes]

time_dt = pd.to_datetime(time, unit='s')
time_num = mdates.date2num(time_dt.to_pydatetime()) # float matplotlib ne

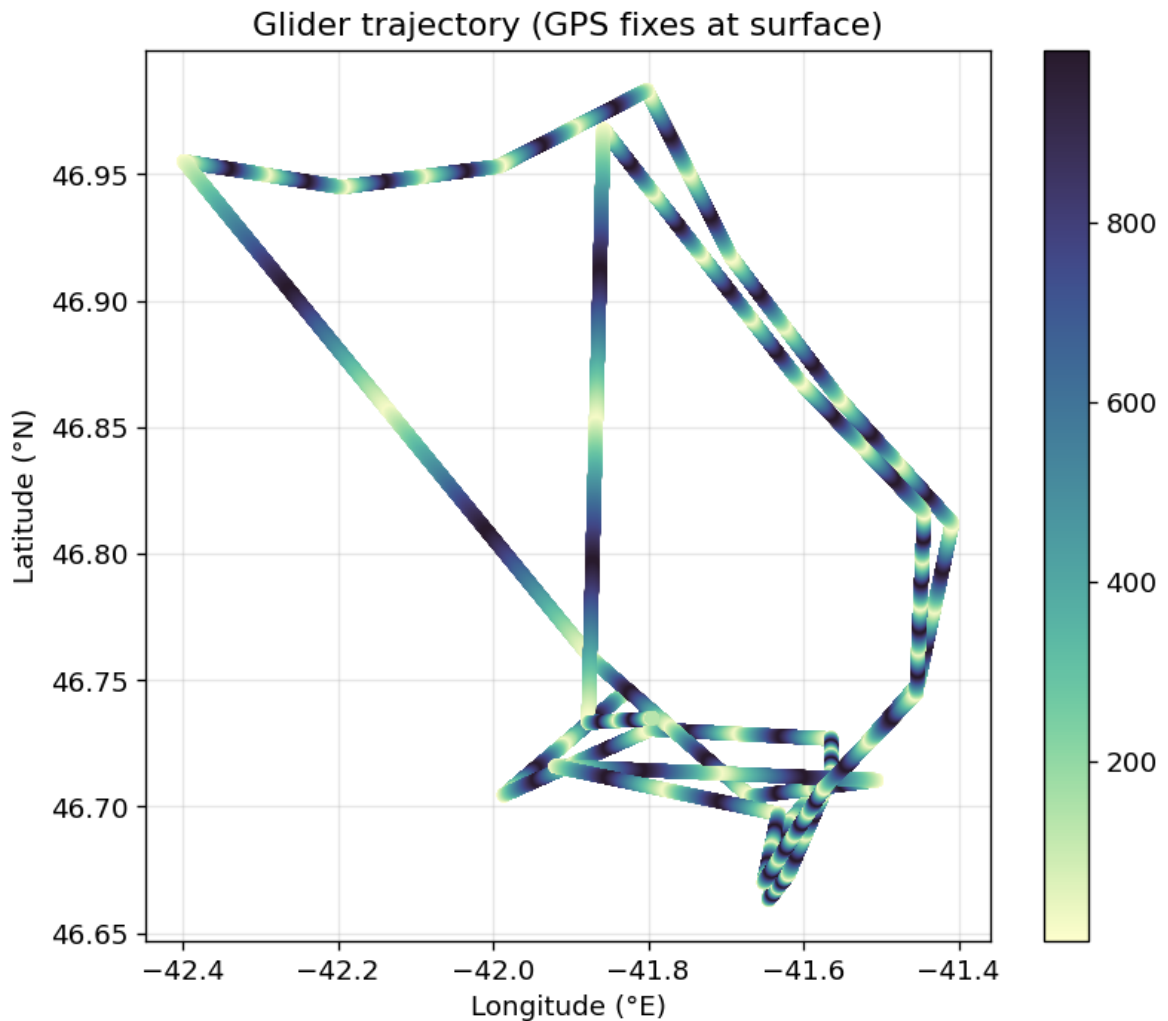
tab10 = plt.get_cmap('tab10')
```

```

cmap66 = ListedColormap([tab10(i % 10) for i in range(66)])

fig, ax = plt.subplots(figsize=(7, 6))
sc = ax.scatter(lon, lat, c=depth, cmap=cmocean.cm.deep, s=20, zorder=3)
ax.plot(lon, lat, lw=0.5, color='grey', zorder=2)
cbar = plt.colorbar(sc, ax=ax)
#cbar.ax.yaxis_date()
#cbar.ax.yaxis.set_major_formatter(mdates.DateFormatter('%b %d'))
ax.set_xlabel('Longitude (°E)')
ax.set_ylabel('Latitude (°N)')
ax.set_title('Glider trajectory (GPS fixes at surface)')
ax.grid(alpha=0.3)
plt.tight_layout()

```



to put trajectory in context, load dataset with geostrophic current in this region/ at this time and visualize

```

In [9]: import copernicusmarine

# already downloaded. dataset for geostrophic current comparison
""" copernicusmarine.subset(
    dataset_id="cmems_obs-sl_glo_phy-ssh_nrt_allsat-l4-duacs-0.125deg_P1D",
    variables=["sla", "ugos", "vgos"],
    minimum_latitude=46.0, maximum_latitude=48.0,
    minimum_longitude=-43.0, maximum_longitude=-41.0,
    start_datetime="2025-08-11T00:00:00",
    end_datetime="2025-08-18T00:00:00",
    output_filename="cmems_sla.nc",

```

```

        output_directory=".",
        force_download=True
    ) """
    ds_cmems = xr.open_dataset("cmems_sla.nc")

```

/home/maranderine/Sync/Studium/STOFF/SoSe26/IMASYS/IMS/lib/python3.14/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
from .autonotebook import tqdm as notebook_tqdm
```

```

In [10]: import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np
import pandas as pd

times = pd.to_datetime(ds_cmems.time.values)
n_days = len(times)

ncols = int(np.ceil(np.sqrt(n_days)))
nrows = int(np.ceil(n_days / ncols))

fig, axes = plt.subplots(
    nrows, ncols,
    figsize=(4*ncols, 4*nrows),
    sharex=True,
    sharey=True
)

axes = np.ravel(axes)

lon_c = ds_cmems.longitude.values
lat_c = ds_cmems.latitude.values

# glider positions
surf_mask = (df['depth'].values < 1)
glat = df['lat_clean'].values[surf_mask]
glon = df['lon_clean'].values[surf_mask]
gtime = pd.to_datetime(df['time'].values[surf_mask], unit='s')

glon_draw = []
glat_draw = []

for ax, t in zip(axes, times):

    day_ds = ds_cmems.sel(time=t)

    sla = day_ds['sla'].values
    u = day_ds['ugos'].values
    v = day_ds['vgos'].values

    cf = ax.contourf(
        lon_c, lat_c, sla,
        levels=20,
        cmap='RdBu_r',
        extend='both'
    )

    ax.streamplot(
        lon_c, lat_c,

```

```
        u, v,
        color='k',
        linewidth=0.5,
        density=1.2,
        arrowsize=0.7
    )

    # Glider positions for that day
    day_mask = gtime.normalize() == pd.Timestamp(t).normalize()

    glon_draw += glon[day_mask].tolist()
    glat_draw += glat[day_mask].tolist()

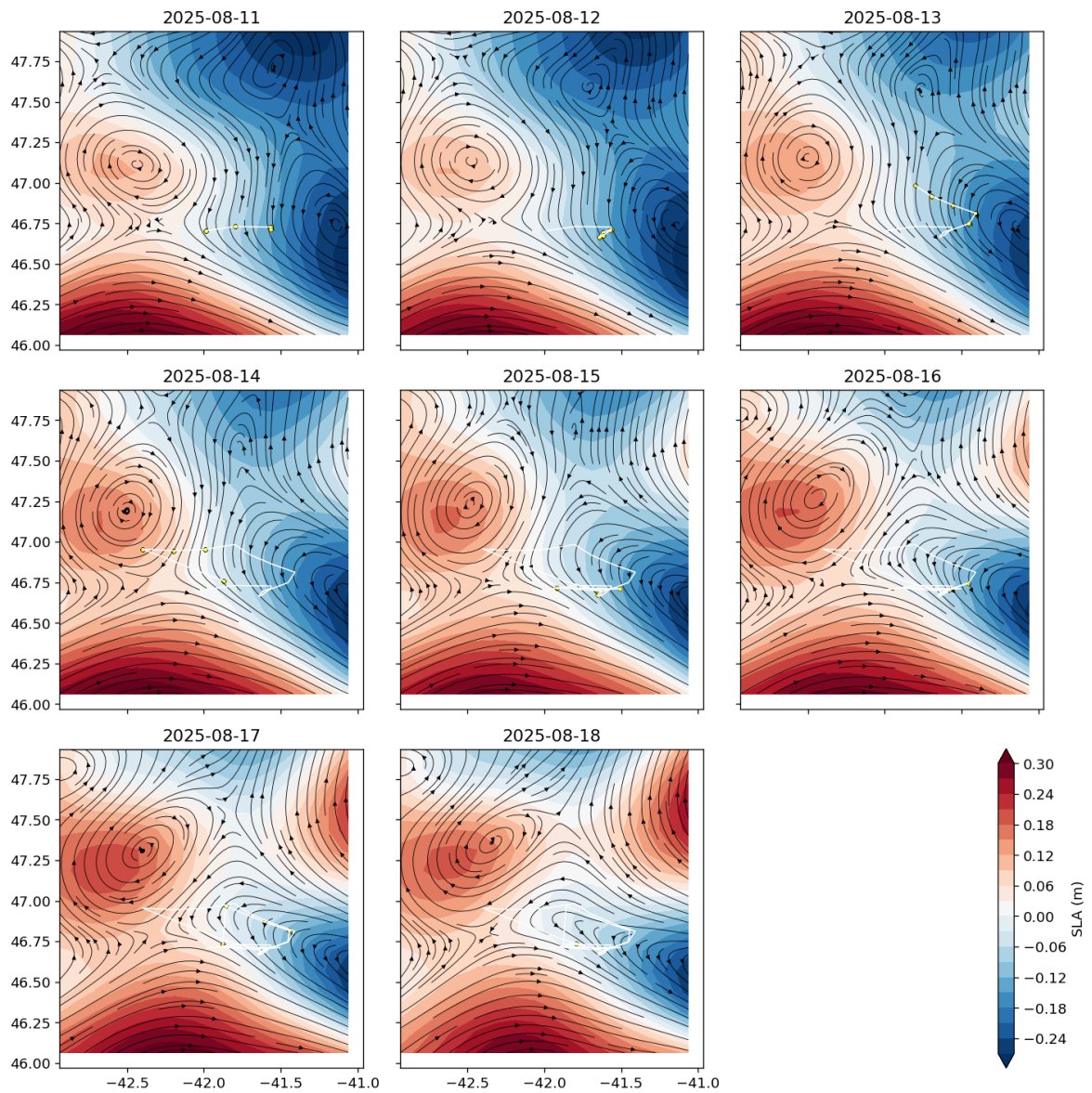
    ax.plot(glon_draw, glat_draw, color='white', lw=1, alpha=1)
    ax.scatter(
        glon[day_mask],
        glat[day_mask],
        s=10,
        c='yellow',
        edgecolors='k',
        linewidths=0.2,
    )

    ax.set_title(pd.Timestamp(t).strftime('%Y-%m-%d'))

    # hide unused axes
    for ax in axes[n_days:]:
        ax.set_visible(False)

    fig.colorbar(cf, ax=axes[-1], label='SLA (m)')

    plt.tight_layout()
    plt.show()
```



```
In [11]: from matplotlib.animation import FuncAnimation
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 7))

def update(i):

    ax.clear()

    t = times[i]
    day_ds = ds_cmems.sel(time=t)

    sla = day_ds['sla'].values
    u = day_ds['ugos'].values
    v = day_ds['vgos'].values

    ax.contourf(
        lon_c, lat_c, sla,
        levels=20,
        cmap='RdBu_r',
```

```
        extend='both'
    )

    ax.streamplot(
        lon_c, lat_c,
        u, v,
        color='k',
        linewidth=0.6,
        density=1.5,
        arrowsize=0.8
    )

    day_mask = (
        gtime.normalize()
        == pd.Timestamp(t).normalize()
    )

    track_mask = gtime <= pd.Timestamp(t) + pd.Timedelta(days=1)

    # trajectory so far
    ax.plot(
        glon[track_mask], glat[track_mask],
        color='black', lw=3, alpha=0.8
    )

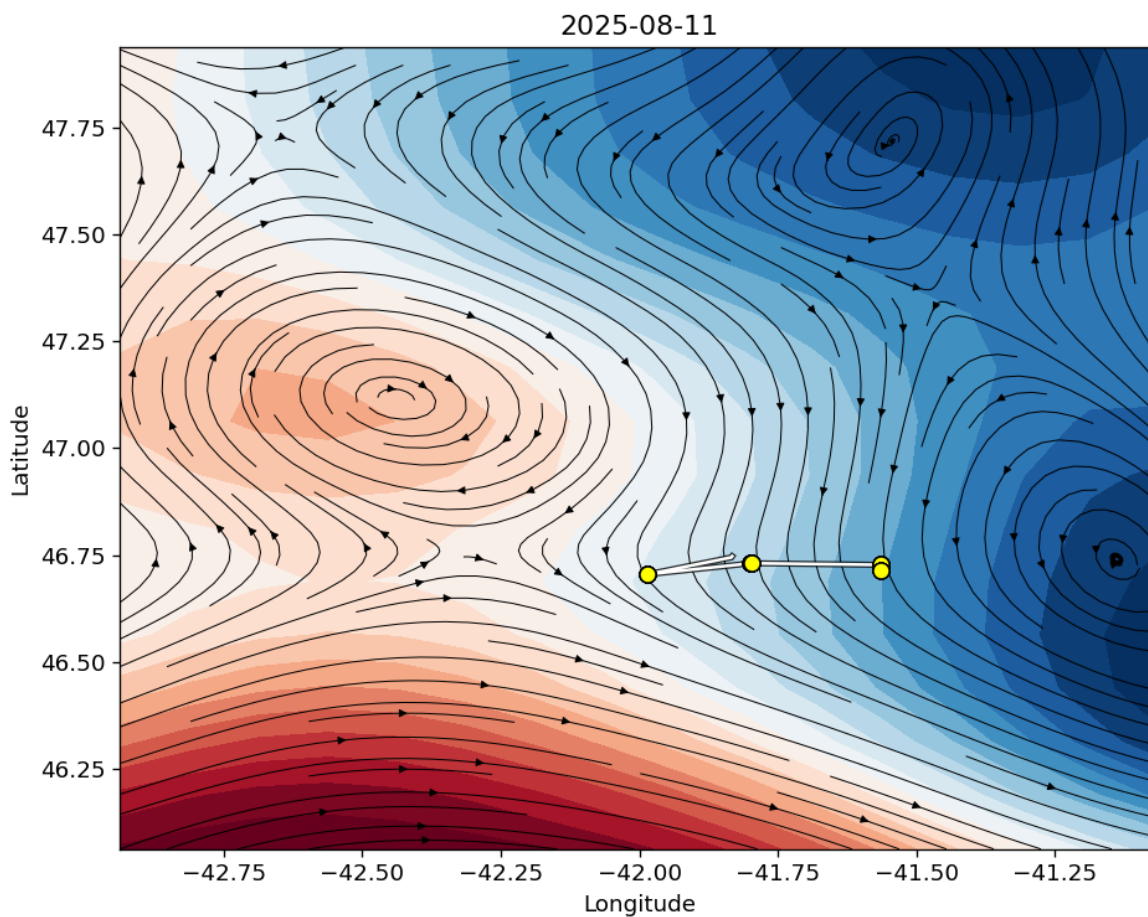
    ax.plot(
        glon[track_mask], glat[track_mask],
        color='white', lw=1.5
    )

    ax.scatter(
        glon[day_mask],
        glat[day_mask],
        s=60,
        c='yellow',
        edgecolors='black',
        linewidths=0.8,
        zorder=10
    )

    ax.set_title(pd.Timestamp(t).strftime('%Y-%m-%d'))
    ax.set_xlabel('Longitude')
    ax.set_ylabel('Latitude')

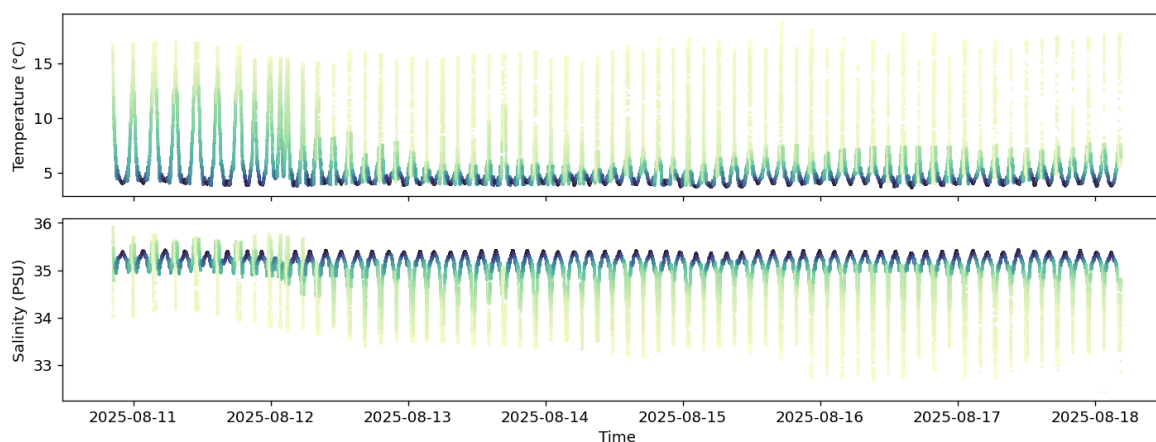
    anim = FuncAnimation(
        fig,
        update,
        frames=len(times),
        interval=1000,
        repeat=True
    )

    anim.save('glider_cmems_animation.gif', writer='pillow')
```



```
In [12]: # Quick sanity plots - raw time series of T and S
fig, axes = plt.subplots(2, 1, figsize=(12, 5), sharex=True)
axes[0].scatter(df['time'], df['temperature'], c=df['depth'], cmap=cmocean.
                s=1, rasterized=True)
axes[0].set_ylabel('Temperature (°C)')
axes[1].scatter(df['time'], df['salinity'], c=df['depth'], cmap=cmocean.cr
                s=1, rasterized=True)
axes[1].set_ylabel('Salinity (PSU)')
axes[1].set_xlabel('Time')
fig.suptitle('Raw time series (colour = depth)')
plt.tight_layout()
```

Raw time series (colour = depth)



Temperature (top panel):

Deep water (dark blue) stays consistently ~4–5°C the whole time — stable bottom water mass Surface temperature (yellow-green) starts at ~15–17°C early on and

drops to $\sim 7\text{--}8^\circ\text{C}$ by the end — the glider moved into progressively cooler surface waters as it traveled, consistent with the westward drift in the trajectory toward the anticyclone boundary

Salinity (bottom panel):

Deep water consistently $\sim 35.0\text{--}35.1$ PSU Surface dips to $\sim 33\text{--}34$ PSU — a fresher surface layer typical of North Atlantic (net precipitation, river outflow from NAC)
The salinity range stays tight ($\sim 33\text{--}36$ PSU), no extreme outliers $\rightarrow$ outlier removal worked

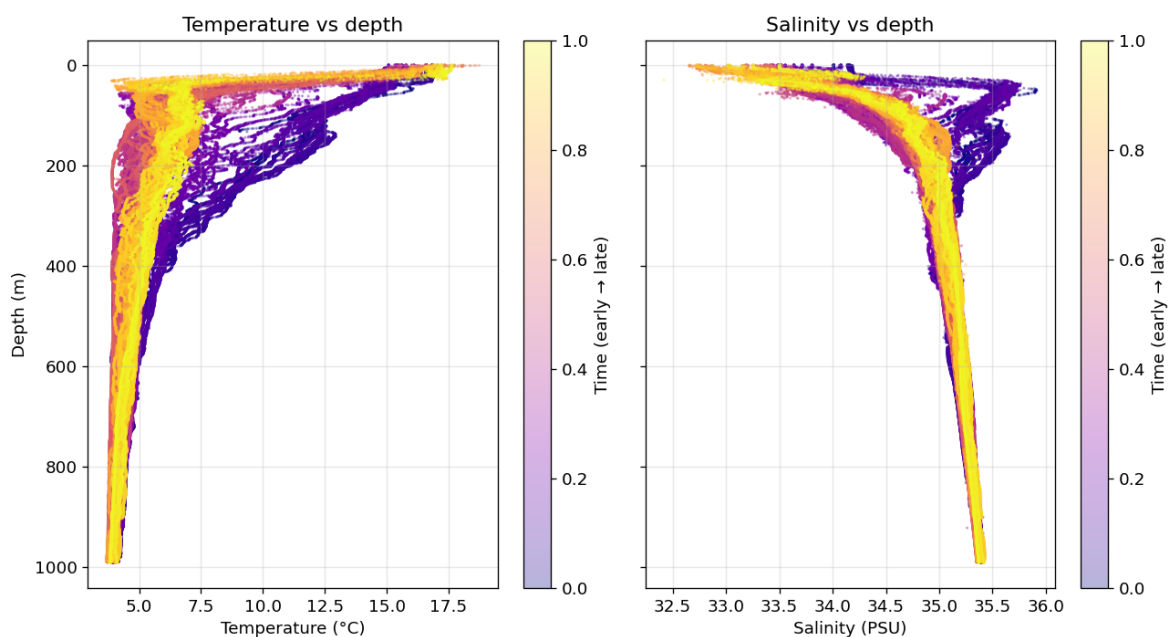
```
In [13]: # T and S vs depth, coloured by time to show evolution along the mission
t_num = df['time'].astype(np.int64)
t_norm = (t_num - t_num.min()) / (t_num.max() - t_num.min())

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 6), sharey=True)

sc1 = ax1.scatter(df['temperature'], df['depth'], c=t_norm, cmap='plasma',
                  s=1, alpha=0.3, rasterized=True)
plt.colorbar(sc1, ax=ax1, label='Time (early → late)')
ax1.set_xlabel('Temperature (°C)')
ax1.set_ylabel('Depth (m)')
ax1.invert_yaxis()
ax1.set_title('Temperature vs depth')
ax1.grid(alpha=0.3)

sc2 = ax2.scatter(df['salinity'], df['depth'], c=t_norm, cmap='plasma',
                  s=1, alpha=0.3, rasterized=True)
plt.colorbar(sc2, ax=ax2, label='Time (early → late)')
ax2.set_xlabel('Salinity (PSU)')
ax2.set_title('Salinity vs depth')
ax2.grid(alpha=0.3)

plt.tight_layout()
```



assigning dive id

```

In [14]: #df = df.copy()  # Take every 10th point for faster dfing

# — Assign dive_id —————

z = df['depth'].values
dz = np.sign(np.diff(z, prepend=z[0]))
dz_smooth = pd.Series(dz).rolling(50, center=True, min_periods=1).median()
going_down = dz_smooth > 0
df['dive_id'] = np.cumsum(np.diff(going_down.astype(int), prepend=0) > 0)
print("dive_id assigned from depth direction changes")
df.loc[df.index[2:], 'dive_id'] -= 1

import matplotlib.pyplot as plt
import numpy as np

dives = sorted(df['dive_id'].unique())
n_dives = len(dives)

ncols = int(np.ceil(np.sqrt(n_dives)))
nrows = int(np.ceil(n_dives / ncols))

fig, axes = plt.subplots(
    nrows,
    ncols,
    figsize=(3 * ncols, 3 * nrows),
    sharey=True
)

axes = np.ravel(axes)

for ax, dive_id in zip(axes, dives):
    dive = df[df['dive_id'] == dive_id

    ax.scatter(
        dive['time'],
        dive['depth'],
        s=5,
        alpha=0.3,
        rasterized=True
    )

    ax.set_title(f'Dive {dive_id}')
    ax.invert_yaxis()

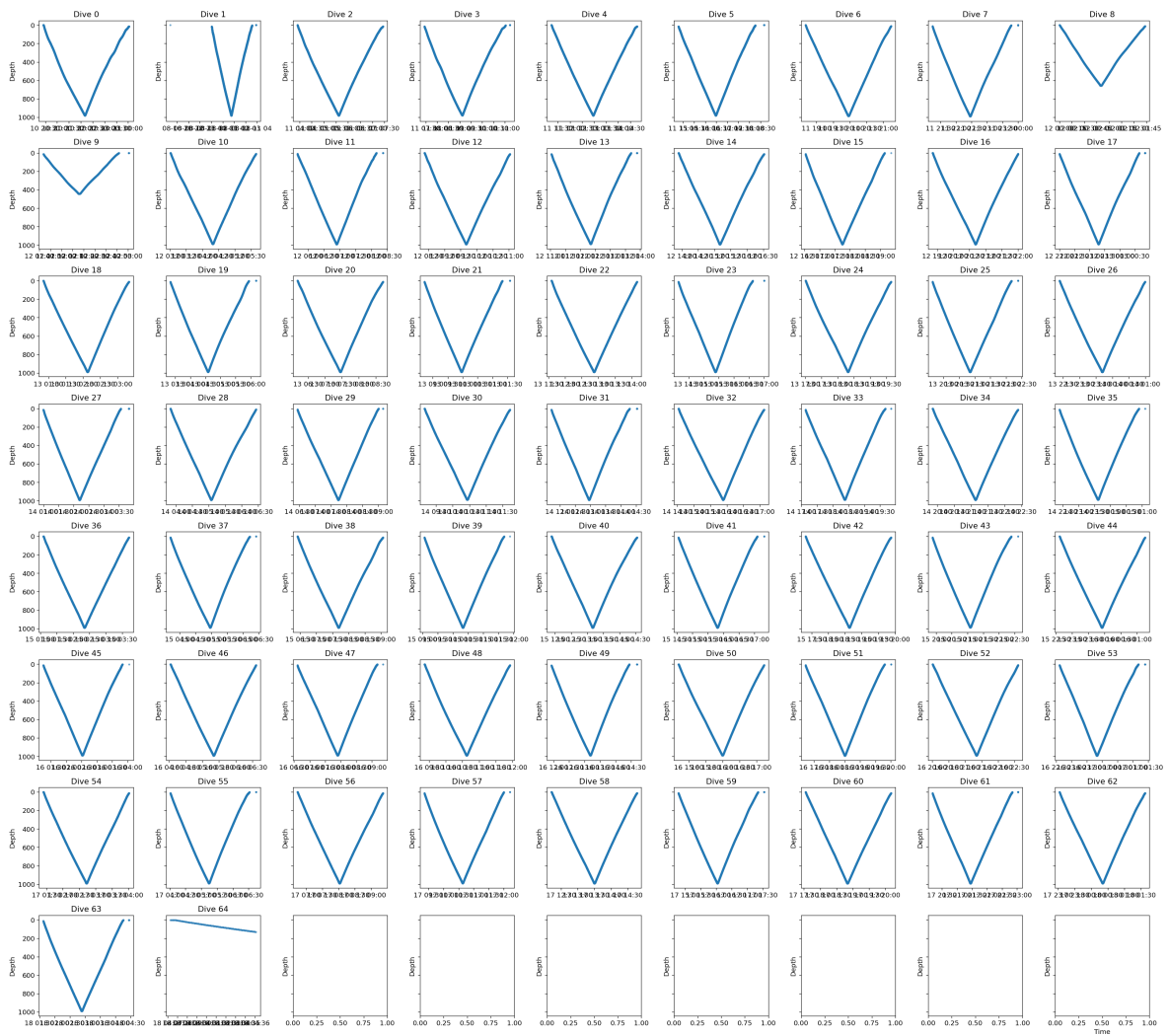
axes[-1].set_xlabel('Time')
for ax in axes:
    ax.set_ylabel('Depth')

plt.tight_layout()
plt.show()

print(f'number of dives identified: {n_dives}')

```

dive_id assigned from depth direction changes



number of dives identified: 65

3 — Preprocessing & derived variables

```
In [15]: window = 11

med = df['temperature'].rolling(
    window=window,
    center=True,
    min_periods=1
).median()

residual = abs(df['temperature'] - med)

threshold = residual.median()

mask = residual < threshold
df_outliers = df[mask]

In [16]: # Kept data (grey) vs removed outliers (red) for temperature and salinity
fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

axes[0].scatter(df['temperature'], df['depth'], s=1, c='steelblue',
                alpha=0.2, rasterized=True, label=f'kept ({len(df):,})')
axes[0].scatter(df_outliers['temperature'], df_outliers['depth'], s=4, c='red',
                alpha=0.6, rasterized=True, label=f'removed ({len(df_outl:
axes[0].set_xlabel('Temperature (°C)'))
```

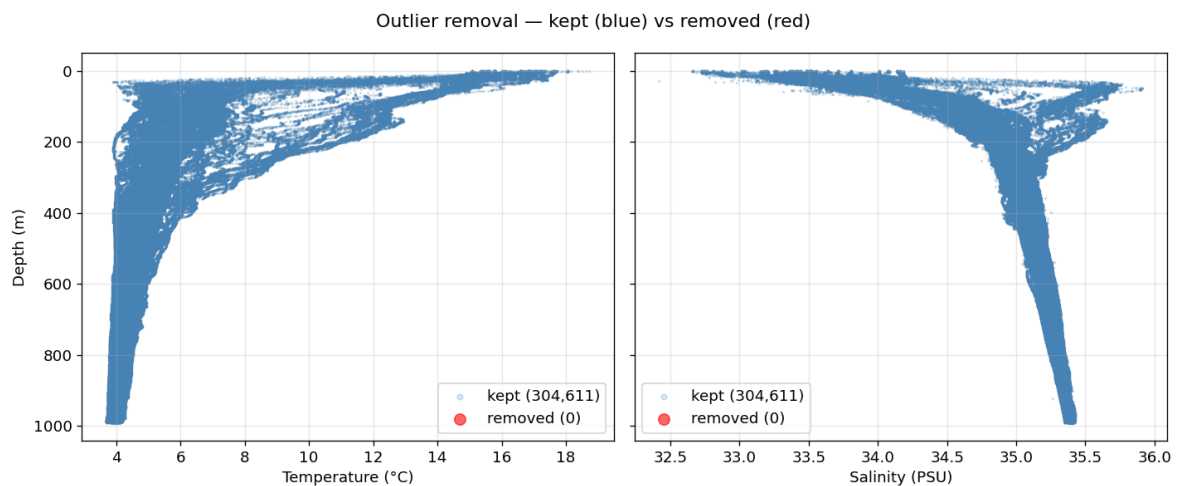
```

axes[0].set_ylabel('Depth (m)')
axes[0].invert_yaxis()
axes[0].legend(markerscale=4)
axes[0].grid(alpha=0.3)

axes[1].scatter(df['salinity'], df['depth'], s=1, c='steelblue',
               alpha=0.2, rasterized=True, label=f'kept ({len(df):,})')
axes[1].scatter(df_outliers['salinity'], df_outliers['depth'], s=4, c='red',
               alpha=0.6, rasterized=True, label=f'removed ({len(df_outliers):,})')
axes[1].set_xlabel('Salinity (PSU)')
axes[1].legend(markerscale=4)
axes[1].grid(alpha=0.3)

fig.suptitle('Outlier removal — kept (blue) vs removed (red)')
plt.tight_layout()

```



```

In [17]: # — Derived variables via gsw (TEOS-10) —————

# Absolute Salinity and Conservative Temperature (TEOS-10 standards)
df['SA'] = gsw.SA_from_SP(df['salinity'], df['pressure'], df['lon'], df['lat'])
df['CT'] = gsw.CT_from_t(df['SA'], df['temperature'], df['pressure'])

# Potential density anomaly  $\sigma_0 = \rho(SA, CT, p=0) - 1000$  [kg/m³]
df['sigma0'] = gsw.sigma0(df['SA'], df['CT'])

# In-situ density (for  $N^2$  calculation)
df['rho'] = gsw.rho(df['SA'], df['CT'], df['pressure'])

print(df[['SA', 'CT', 'sigma0', 'rho']].describe().round(4))

```

	SA	CT	sigma0	rho
count	304611.0000	304611.0000	304611.0000	304611.0000
mean	35.2302	5.4933	27.6350	1029.8888
std	0.4026	2.4993	0.6287	1.8318
min	32.5723	3.6020	23.5812	1023.5844
25%	35.1797	4.1545	27.5547	1028.6474
50%	35.3485	4.6391	27.8367	1030.0668
75%	35.4604	5.5599	27.9852	1031.3805
max	36.0840	18.7785	28.1255	1032.6927

4 — Profile plotter (single dive)

```

In [18]: def plot_profile(df, dive_id, smooth_sigma=2):
          """T, S,  $\sigma_0$  vs depth for one dive."""

```

```

d = df[df['dive_id'] == dive_id].sort_values('depth')

T = gaussian_filter1d(d['temperature'].values, smooth_sigma)
S = gaussian_filter1d(d['salinity'].values, smooth_sigma)
sig = gaussian_filter1d(d['sigma0'].values, smooth_sigma)
z = d['depth'].values

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(10, 7), sharey=True)

ax1.plot(T, z, color=cmocean.cm.thermal(0.7), lw=1.5)
ax1.set_xlabel('Temperature (°C)')
ax1.set_ylabel('Depth (m)')
ax1.invert_yaxis()
ax1.grid(alpha=0.3)

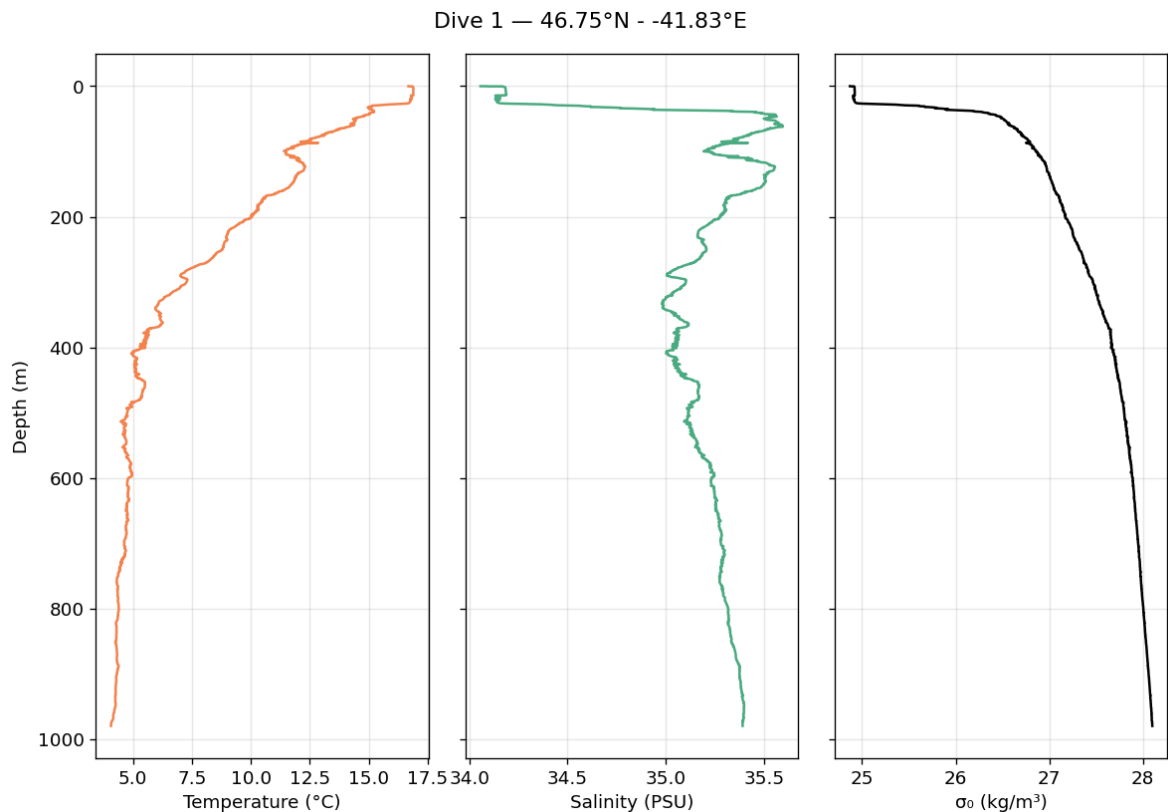
ax2.plot(S, z, color=cmocean.cm.haline(0.6), lw=1.5)
ax2.set_xlabel('Salinity (PSU)')
ax2.grid(alpha=0.3)

ax3.plot(sig, z, color='k', lw=1.5)
ax3.set_xlabel('σ0 (kg/m³)')
ax3.grid(alpha=0.3)

lat = d['lat'].iloc[0]
lon = d['lon'].iloc[0]
fig.suptitle(f'Dive {dive_id} - {lat:.2f}°N - {lon:.2f}°E')
plt.tight_layout()
return fig

# — Change dive_id to inspect different dives —————
DIVE_TO_PLOT = 1
fig = plot_profile(df, DIVE_TO_PLOT)

```



5 — Section plotter (2D transect)

```
In [19]: x_vals = df.drop_duplicates('lat')
df.loc[df['dive_id'] == 1]
```

```
Out [19]:
```

	time	lat	lon	depth	pressure	tempera
0	2025-08-10 20:24:56.413000107	46.746972	-41.832464	0.091766	0.260000	16.598
1	2025-08-10 20:24:58.413000107	46.744477	-41.833857	0.091766	0.260000	16.596
6514	2025-08-11 00:02:04.358000040	46.712344	-41.912126	14.188104	14.630001	16.565
6515	2025-08-11 00:02:06.358000040	46.712351	-41.912147	14.510675	14.900000	16.571
6516	2025-08-11 00:02:08.358000040	46.712356	-41.912164	15.025199	15.250000	16.569
...	...	...	...	...	...	...
12878	2025-08-11 03:54:22.398000002	46.704492	-41.986128	0.356522	0.300000	16.843
12879	2025-08-11 03:54:24.398000002	46.704508	-41.986131	0.350967	0.290000	16.842
12880	2025-08-11 03:54:26.398000002	46.704520	-41.986131	0.355254	0.300000	16.843
12881	2025-08-11 03:54:28.398000002	46.704536	-41.986128	0.366369	0.310000	16.842
12882	2025-08-11 03:54:30.397000074	46.704547	-41.986125	0.385172	0.360000	16.843

6371 rows × 14 columns

```
In [20]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binned_statistic_2d

def make_section_grid(
    df,
    var,
    depth_bins=None,
    x_bins=None,
    x_col='lat'
):
    """
    Bin data onto a regular depth × x grid using scipy.

    Parameters
    -----
    df : pandas.DataFrame
    var : str
```

```

        Variable to average into bins.
    depth_bins : array-like
    x_bins : array-like
    x_col : str
        Usually 'lat', 'lon', or distance.

Returns
-----
grid : ndarray
    Shape (n_depth_bins-1, n_x_bins-1)
depth_centres : ndarray
x_centres : ndarray
"""

if depth_bins is None:
    depth_bins = np.arange(
        0,
        df['depth'].max() + 25,
        25
    )

if x_bins is None:
    x_bins = np.linspace(
        df[x_col].min(),
        df[x_col].max(),
        100
    )

mask = (
    np.isfinite(df[x_col]) &
    np.isfinite(df['depth']) &
    np.isfinite(df[var])
)

grid, x_edges, depth_edges, _ = binned_statistic_2d(
    df.loc[mask, x_col],
    df.loc[mask, 'depth'],
    df.loc[mask, var],
    statistic='mean',
    bins=[x_bins, depth_bins]
)

# scipy returns (x_bin, depth_bin)
# pcolormesh expects (depth, x)
grid = grid.T

x_centres = 0.5 * (x_edges[:-1] + x_edges[1:])
depth_centres = 0.5 * (depth_edges[:-1] + depth_edges[1:])

return grid, depth_centres, x_centres


def plot_section(
    df,
    var,
    cmap,
    label,
    vmin=None,
    vmax=None,
    depth_bins=None,

```

```

x_bins=None,
x_col='lat',
x_label='Latitude (°N)'
):

grid, depths, x_vals = make_section_grid(
    df=df,
    var=var,
    depth_bins=depth_bins,
    x_bins=x_bins,
    x_col=x_col
)

fig, ax = plt.subplots(figsize=(12, 5))

im = ax.pcolormesh(
    x_vals,
    depths,
    grid,
    cmap=cmocean.cm.__dict__.get(cmap, cmap),
    vmin=vmin,
    vmax=vmax,
    shading='auto'
)

cbar = plt.colorbar(
    im,
    ax=ax,
    label=label,
    pad=0.02
)

ax.set_xlabel(x_label)
ax.set_ylabel('Depth (m)')
ax.invert_yaxis()

ax.set_title(
    f'{label} - Atlantic section'
)

plt.tight_layout()

return fig, ax

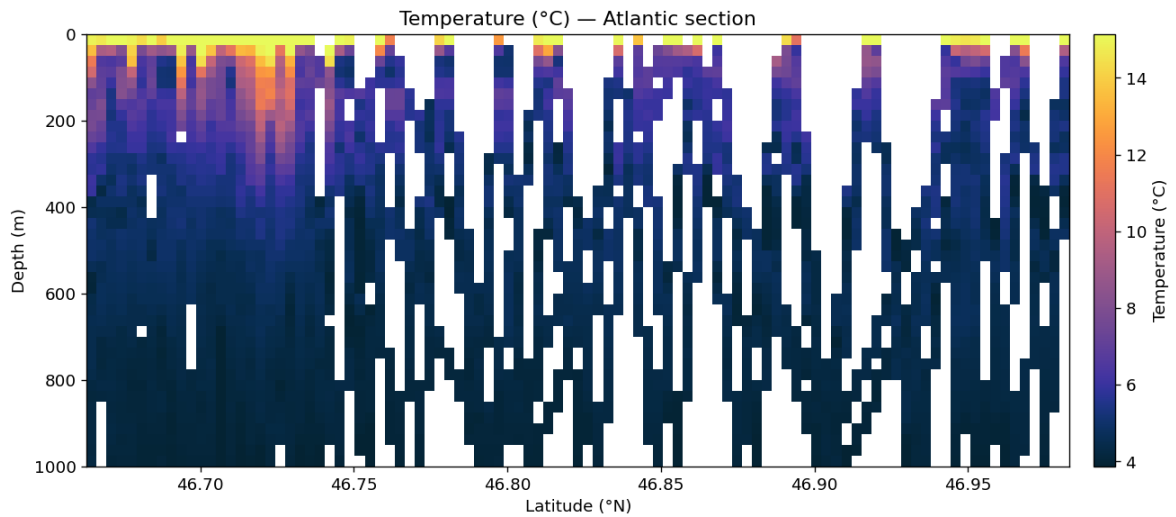
```

```

In [21]: fig, ax = plot_section(
    df,
    var='temperature',
    cmap='thermal',
    label='Temperature (°C)',
    vmin=df['temperature'].quantile(0.02),
    vmax=df['temperature'].quantile(0.98),

    x_col='lat_clean',
    x_label='Latitude (°N)'
)

```



```
In [22]: # Haversine distance using numpy
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    dlat = np.radians(lat2 - lat1)
    dlon = np.radians(lon2 - lon1)
    a = np.sin(dlat/2)**2 + np.cos(np.radians(lat1)) * np.cos(np.radians(lat2)) * np.sin(dlon/2)**2
    return R * 2 * np.arcsin(np.sqrt(a))

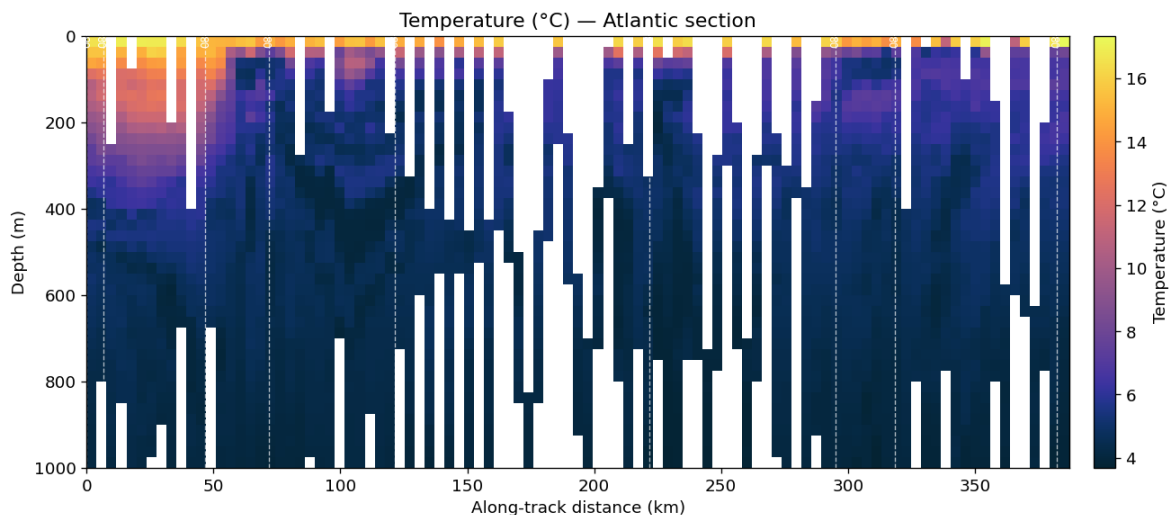
lats = df['lat_clean'].values
lons = df['lon_clean'].values

dists = haversine_km(lats[:-1], lons[:-1], lats[1:], lons[1:])
df['distance_km'] = np.concatenate([[0], np.cumsum(dists)])

fig, ax = plot_section(df, 'temperature', 'thermal', 'Temperature (°C)',
                      x_col='distance_km', x_label='Along-track distance')

# Add vertical lines for each day
df['date'] = df['time'].dt.date
daily_dist = df.groupby('date')['distance_km'].first()

for date, dist in daily_dist.items():
    ax.axvline(dist, color='white', lw=0.8, linestyle='--', alpha=0.7)
    ax.text(dist, 20, str(date)[5:], color='white', fontsize=7,
           ha='center', rotation=90)
```

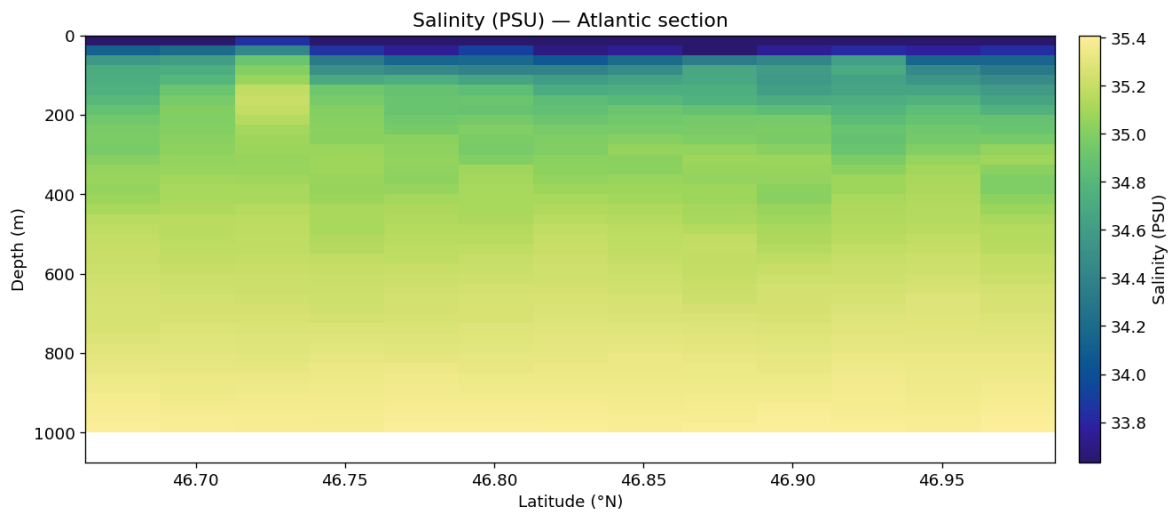


```
In [23]: fig, ax = plot_section(
```

```

df,
var='salinity',
cmap='haline',
label='Salinity (PSU) ',
vmin=df['salinity'].quantile(0.02),
vmax=df['salinity'].quantile(0.98),
depth_bins=np.arange(0, 1100, 25),
x_bins=np.arange(
    df['lat_clean'].min(),
    df['lat_clean'].max() + 0.025,
    0.025
),
x_col='lat_clean',
x_label='Latitude (°N) '
)

```



```

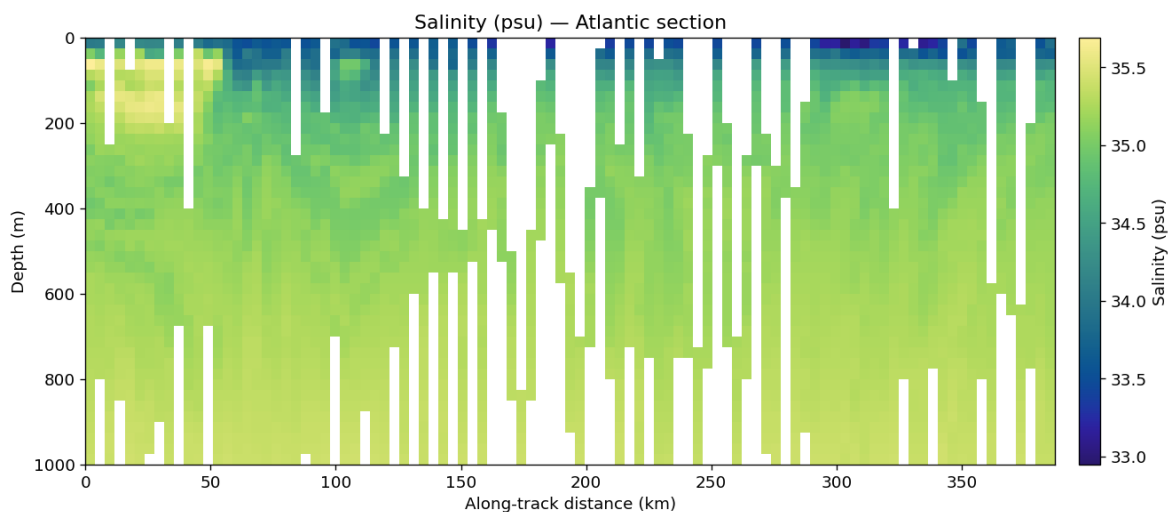
In [24]: plot_section(df, 'salinity', 'haline', 'Salinity (psu)',
                    x_col='distance_km', x_label='Along-track distance (km)')

```

```

Out[24]: (<Figure size 1440x600 with 2 Axes>,
<Axes: title={'center': 'Salinity (psu) — Atlantic section'}, xlabel='Along-track distance (km)', ylabel='Depth (m)'>)

```



```

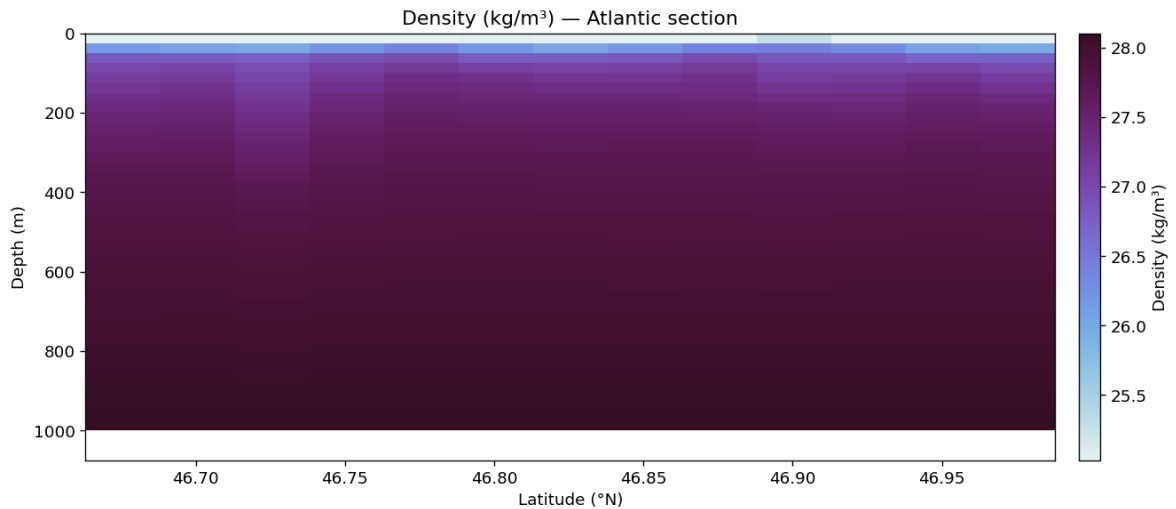
In [25]: fig, ax = plot_section(
    df,
    var='sigma0',
    cmap='dense',
    label='Density (kg/m³) ',
    vmin=df['sigma0'].quantile(0.02),

```

```

vmax=df['sigma0'].quantile(0.98),
depth_bins=np.arange(0, 1100, 25),
x_bins=np.arange(
    df['lat_clean'].min(),
    df['lat_clean'].max() + 0.025,
    0.025
),
x_col='lat_clean',
x_label='Latitude (°N)'
)

```



```

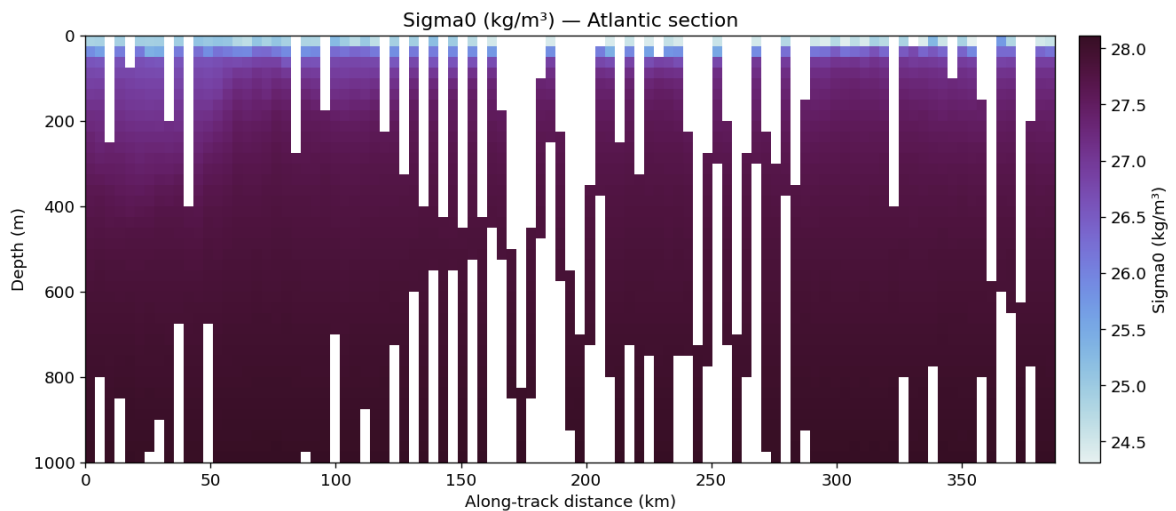
In [26]: plot_section(df, 'sigma0', 'dense', 'Sigma0 (kg/m³)',
                    x_col='distance_km', x_label='Along-track distance (km)')

```

```

Out[26]: (<Figure size 1440x600 with 2 Axes>,
<Axes: title={'center': 'Sigma0 (kg/m³) — Atlantic section'}, xlabel='Al
ong-track distance (km)', ylabel='Depth (m)'>)

```



6 — T-S diagram (water mass identification)

```

In [27]: # — Isopycnal contours ( $\sigma_\theta$  lines) —
T_range = np.linspace(df['CT'].min() - 1, df['CT'].max() + 1, 200)
S_range = np.linspace(df['SA'].min() - 0.2, df['SA'].max() + 0.2, 200)
T_grid, S_grid = np.meshgrid(T_range, S_range)
sigma_grid = gsw.sigma0(S_grid, T_grid)

fig, ax = plt.subplots(figsize=(7, 7))

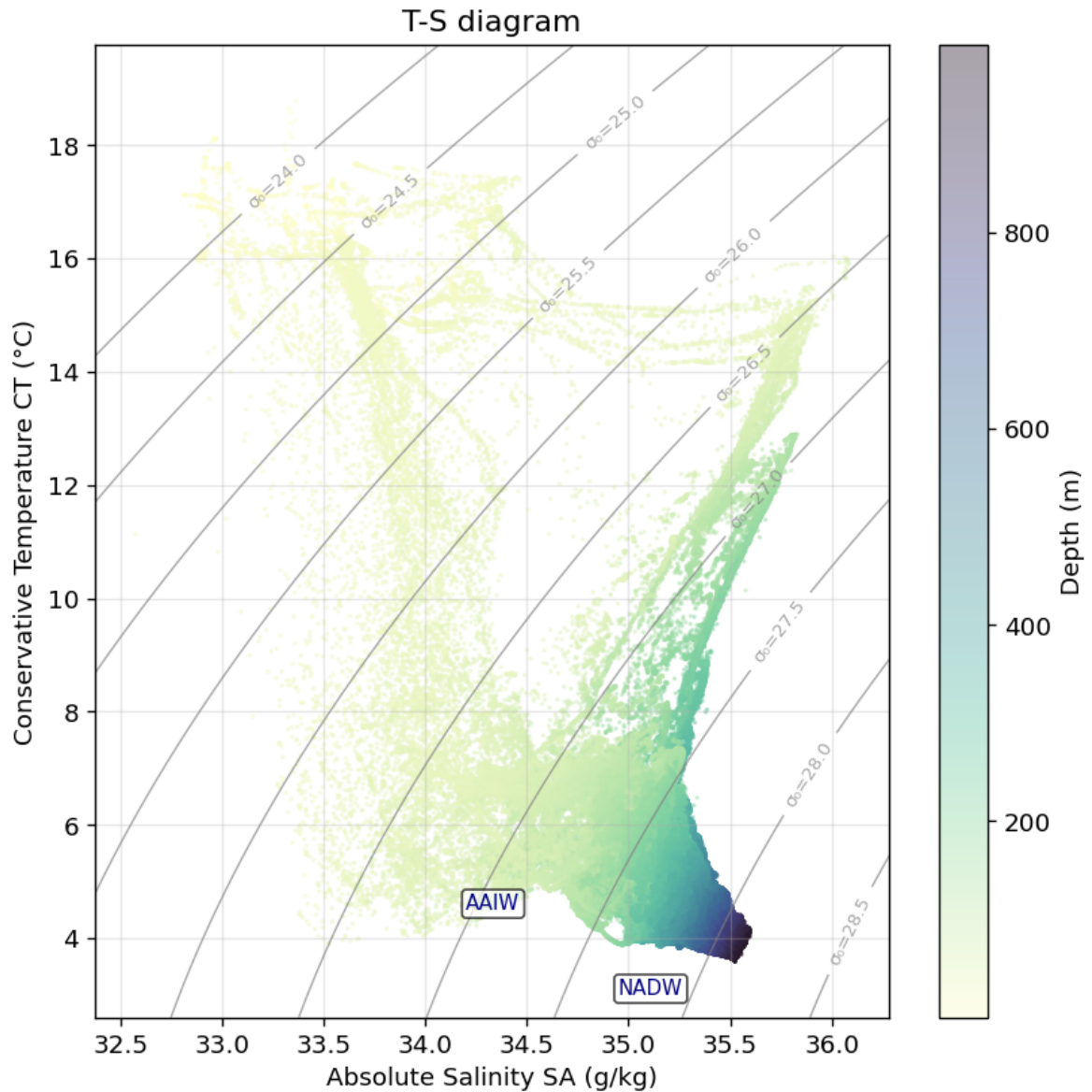
```

```
# Scatter coloured by depth
sc = ax.scatter(
    df['SA'], df['CT'],
    c=df['depth'], cmap=cmocean.cm.deep,
    s=1, alpha=0.4, rasterized=True
)
plt.colorbar(sc, ax=ax, label='Depth (m)')

# Isopycnals
sigma_levels = np.arange(24, 29, 0.5)
cs = ax.contour(S_grid, T_grid, sigma_grid,
                levels=sigma_levels, colors='grey', linewidths=0.8, alpha=
ax.clabel(cs, fmt='σθ=%.1f', fontsize=8)

# Water mass labels (approximate Atlantic values - adjust to your data)
water_masses = {
    'AAIW': (34.2, 4.5),      # S~34.2, T~4.5°C, ~800m
    'NADW': (34.95, 3.0),    # S~34.95, T~3°C, ~2000m
    'AABW': (34.68, 0.5),    # S~34.68, T<1°C, >4000m
}
for name, (s, t) in water_masses.items():
    ax.annotate(name, xy=(s, t), fontsize=9, color='navy',
                bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.7)

ax.set_xlabel('Absolute Salinity SA (g/kg)')
ax.set_ylabel('Conservative Temperature CT (°C)')
ax.set_title('T-S diagram')
ax.grid(alpha=0.3)
plt.tight_layout()
```



7 — Buoyancy frequency N^2

```
In [28]: def compute_N2_profile(dive_df):
    """Compute  $N^2$  for one dive using gsw.Nsquared."""
    d = dive_df.sort_values('pressure')
    lat = d['lat'].values
    N2, p_mid = gsw.Nsquared(
        d['SA'].values,
        d['CT'].values,
        d['pressure'].values,
        lat=lat,
        # full N-element array
    )
    lat_mid = 0.5 * (lat[:-1] + lat[1:]) # N-1 midpoint lats for z conve
    z_mid = -gsw.z_from_p(p_mid, lat_mid)
    return N2, z_mid

def plot_N2_profile(df, dive_id):
    d = df[df['dive_id'] == dive_id]
    N2, z_mid = compute_N2_profile(d)

    N2_smooth = gaussian_filter1d(np.maximum(N2, 0), sigma=3)
```

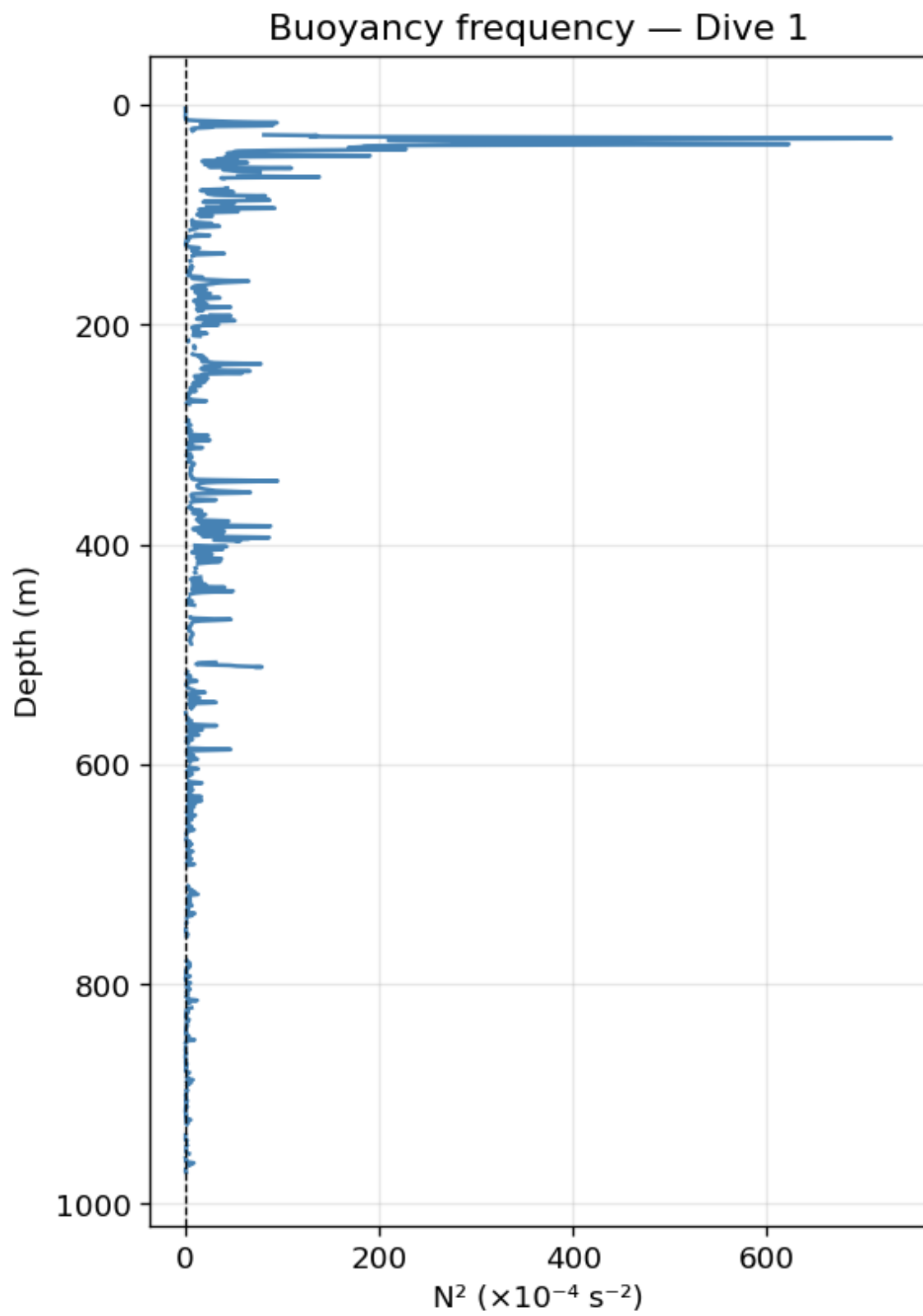
```
fig, ax = plt.subplots(figsize=(5, 7))
ax.plot(N2_smooth * 1e4, z_mid, color='steelblue', lw=1.5,
        label='N2 (×10-4 s-2)')
ax.axvline(0, color='k', lw=0.8, ls='--')
ax.set_xlabel('N2 (×10-4 s-2)')
ax.set_ylabel('Depth (m)')
ax.invert_yaxis()
ax.set_title(f'Buoyancy frequency - Dive {dive_id}')
ax.grid(alpha=0.3)
plt.tight_layout()
return fig
```

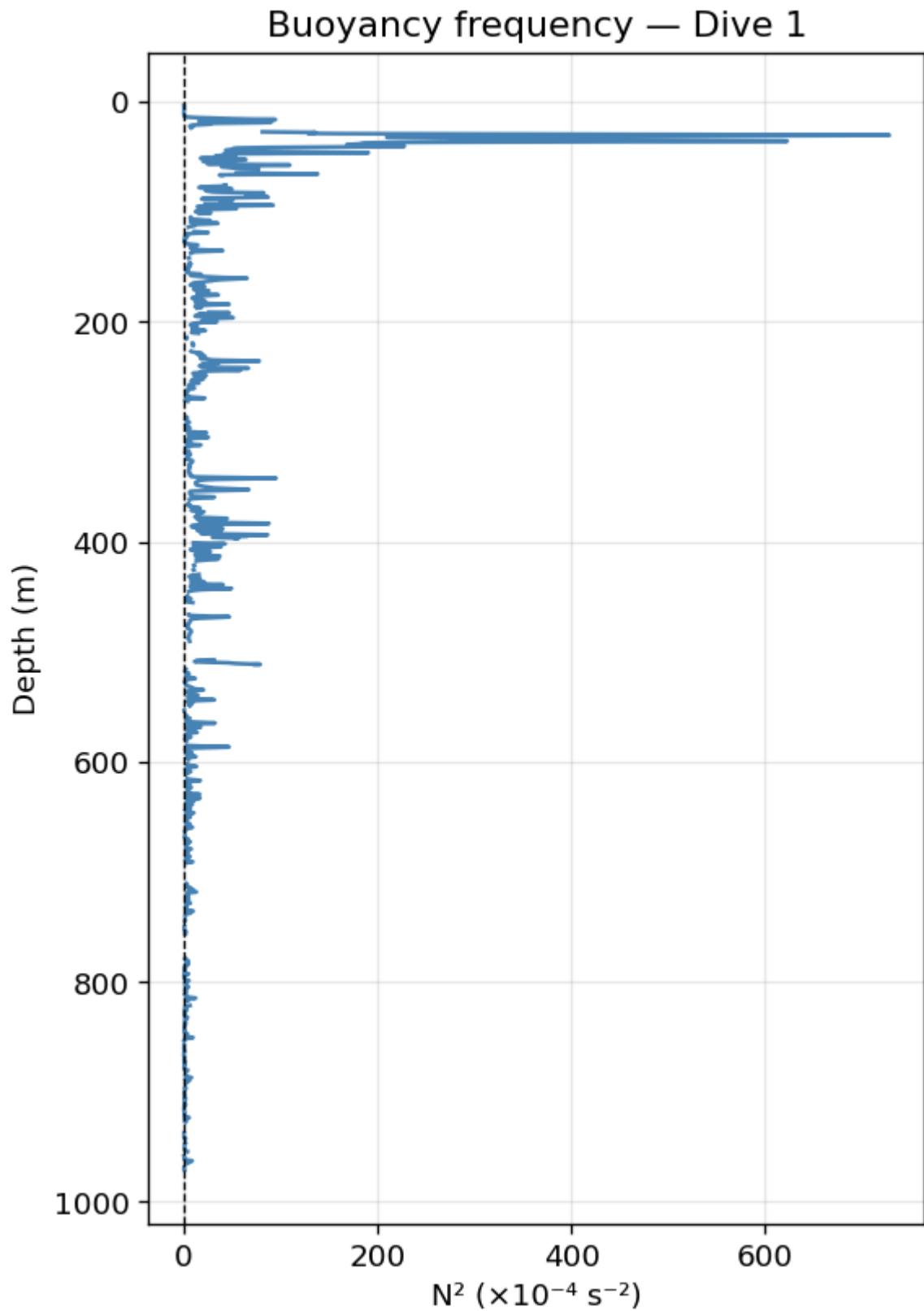
```
plot_N2_profile(df, DIVE_TO_PLOT)
```

```
/home/maranderine/Sync/Studium/STOFF/SoSe26/IMASYS/IMS/lib/python3.14/site-
packages/gsw/stability.py:86: RuntimeWarning: divide by zero encountered
in divide
```

```
N2 = ((g_local**2) / (specvol_mid * db_to_pa * dp))
```

Out [28]:





8 — Save figures

Uncomment to export all figures for the report.

```
In [29]: # import os
# os.makedirs('figures', exist_ok=True)
# for i, fig in enumerate(plt.get_fignums()):
#     plt.figure(fig).savefig(f'figures/fig_{i:02d}.pdf', bbox_inches='tight')
```

